

LAB PROGRAMS

Name: Preethika M

Reg No: 192421084

Course code: CSA5162

Course Name: Cryptography and Network Security

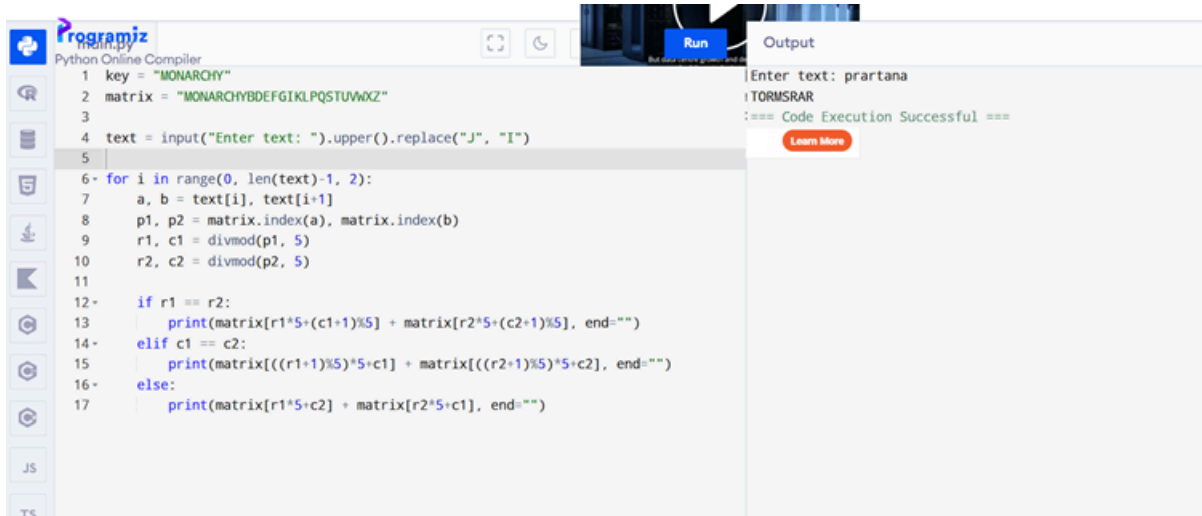
1. Write a Python program for Caesar cipher involves replacing each letter of the alphabet with the letter standing k places further down the alphabet, for k in the range 1 through 25.

main.py	Output
<pre>1 text = input("Enter message: ") 2 key = int(input("Enter key: ")) 3 4 result = "" 5 6 for ch in text: 7 if ch.isalpha(): 8 result += chr((ord(ch.upper()) - 65 + key) % 26 + 65) 9 else: 10 result += ch 11 12 print("Encrypted Text:", result)</pre>	<pre>Enter message: break Enter key: 3 Encrypted Text: EUHDN === Code Execution Successful ===</pre>

2. Write a Python program for monoalphabetic substitution cipher maps a plaintext alphabet to a ciphertext alphabet, so that each letter of the plaintext alphabet maps to a single unique letter of the ciphertext alphabet.

main.py	Output
<pre>1 plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ" 2 cipher = "QWERTYUIOPASDFGHJKLZXCVBNM" 3 4 text = input("Enter plaintext: ").upper() 5 result = "" 6 7 for ch in text: 8 if ch in plain: 9 result += cipher[plain.index(ch)] 10 else: 11 result += ch 12 13 print("Ciphertext:", result)</pre>	<pre>Enter plaintext: break Ciphertext: WKTQA === Code Execution Successful ===</pre>

3. Write a Python program for Playfair algorithm is based on the use of a 5 X 5 matrix of letters constructed using a keyword. Plaintext is encrypted two letters at a time using this matrix.



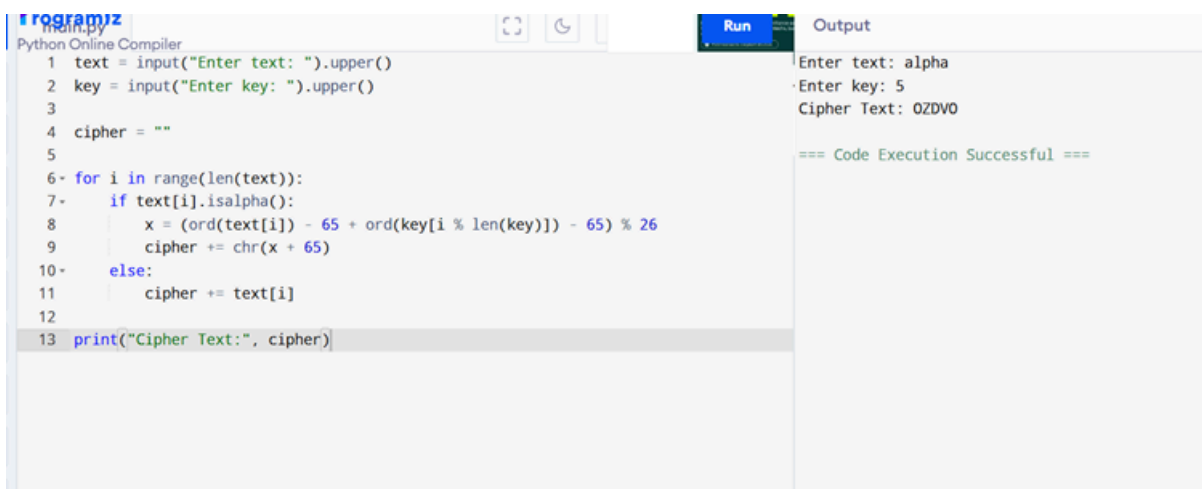
```

1 key = "MONARCHY"
2 matrix = "MONARCHYBDEFGIKLPQSTUVWXZ"
3
4 text = input("Enter text: ").upper().replace("J", "I")
5
6 for i in range(0, len(text)-1, 2):
7     a, b = text[i], text[i+1]
8     p1, p2 = matrix.index(a), matrix.index(b)
9     r1, c1 = divmod(p1, 5)
10    r2, c2 = divmod(p2, 5)
11
12    if r1 == r2:
13        print(matrix[r1*5+(c1+1)%5] + matrix[r2*5+(c2+1)%5], end=" ")
14    elif c1 == c2:
15        print(matrix[((r1+1)%5)*5+c1] + matrix[((r2+1)%5)*5+c2], end=" ")
16    else:
17        print(matrix[r1*5+c2] + matrix[r2*5+c1], end=" ")

```

Output: Enter text: prartana
TORMSRAR
=== Code Execution Successful ===

4. Write a C program for polyalphabetic substitution cipher uses a separate monoalphabetic substitution cipher for each successive letter of plaintext, depending on a key.



```

1 text = input("Enter text: ").upper()
2 key = input("Enter key: ").upper()
3
4 cipher = ""
5
6 for i in range(len(text)):
7     if text[i].isalpha():
8         x = (ord(text[i]) - 65 + ord(key[i % len(key)]) - 65) % 26
9         cipher += chr(x + 65)
10    else:
11        cipher += text[i]
12
13 print("Cipher Text:", cipher)

```

Output: Enter text: alpha
Enter key: 5
Cipher Text: OZDVO
=== Code Execution Successful ===

5. Write a Python program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p , substitute the ciphertext letter C : $C = E([a, b], p) = (ap + b) \bmod 26$. A basic requirement of any encryption algorithm is that it be one-to-one. That is, if $p \neq q$, then $E(k, p) \neq E(k, q)$. Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a . For example, for $a = 2$ and $b = 3$, then $E([a, b], 0) = E([a, b], 13) = 3$. a. Are there any limitations on the value of b ? b. Determine which values of a are not allowed.

```

Python Online Compiler
1 text = input("Enter text: ").upper()
2 a = int(input("Enter a: "))
3 b = int(input("Enter b: "))
4
5 cipher = ""
6
7- for ch in text:
8-     if ch.isalpha():
9         p = ord(ch) - 65
10        c = (a * p + b) % 26
11        cipher += chr(c + 65)
12
13 print("Cipher Text:", cipher)

```

```

Enter text: welcome
Enter a: 5
Enter b: 2
Cipher Text: IWFUKW

=== Code Execution Successful ===

```

6. Write a C program for ciphertext has been generated with an affine cipher. The most frequent letter of the ciphertext is “B,” and the second most frequent letter of the ciphertext is “U.” Break this code.

```

Python Online Compiler
1 c1 = ord(input("Enter most frequent ciphertext letter: ").upper()) - ord('A')
2 c2 = ord(input("Enter second most frequent ciphertext letter: ").upper()) - ord('A')
3
4 p1 = ord('E') - ord('A')
5 p2 = ord('T') - ord('A')
6
7- for a in range(1, 26):
8-     for b in range(26):
9-         if (a * p1 + b) % 26 == c1 and (a * p2 + b) % 26 == c2:
10            print("Key found:")
11            print("a =", a)
12            print("b =", b)

```

```

Enter most frequent ciphertext letter: b
Enter second most frequent ciphertext letter: u
Key found:
a = 3
b = 15

=== Code Execution Successful ===

```

7. Write a Python program for the following ciphertext was generated using a simple substitution algorithm.

53†††305))6*;4826)4†.)4†);806*;48†8¶(60))85;;]8*;;†*8†83
 (88)5*†;46(;88*96*?;8)*†(;485);5*†2:*†(;4956*2(5*—4)8¶8*
 ;4069285);)6†8)4††;1(†9;48081;8:8†1;48†85;4)485†528806*81
 (†9;48;(88;4(†?34;48)4†;161;;188;†?; Decrypt this message.

1. As you know, the most frequently occurring letter in English is e. Therefore, the first or second (or perhaps third?) most common character in the message is likely to stand for e. Also, e is often seen in pairs (e.g., meet, fleet, speed, seen, been, agree, etc.). Try to find a character in the ciphertext that decodes to e. 2. The most common word in English is “the.” Use this fact to guess the characters that stand for t and h. 3. Decipher the rest of the message by deducing additional words.

```

Python Online Compiler
1 cipher = input("Enter ciphertext: ")
2
3 freq = {}
4
5- for ch in cipher:
6-     if ch != ' ' and ch != '\n':
7         freq[ch] = freq.get(ch, 0) + 1
8
9 print("\nCharacter Frequencies:")
10- for ch, count in sorted(freq.items(), key=lambda x: x[1], reverse=True):
11     print(ch, ":", count)

```

```

Enter ciphertext: annotate
Character Frequencies:
a : 2
n : 2
t : 2
o : 1
e : 1

=== Code Execution Successful ===

```

8. Write a Python program for monoalphabetic cipher is that both sender and receiver must commit the permuted cipher sequence to memory. A common technique for avoiding this is to use a keyword from which the cipher sequence can be generated. For example, using the keyword CIPHER, write out the keyword followed by unused letters in normal order and match this against the plaintext letters: plain: a b c d e f g h i j k l m n o p q r s t u v w x y z cipher: C I P H E R A B D F G J K L M N O Q S T U V W X Y Z

```

1 plain = "abcdefghijklmnopqrstuvwxyz"
2 cipher = "CIPHERABDFGJKLMNOPQRSTUVWXYZ"
3
4 text = input("Enter plaintext: ").lower()
5 result = ""
6
7 for ch in text:
8     if ch in plain:
9         result += cipher[plain.index(ch)]
10    else:
11        result += ch
12
13 print("Ciphertext:", result)

```

Enter plaintext: deal
Ciphertext: HECJ
=== Code Execution Successful ===

9. Write a Python program for PT-109 American patrol boat, under the command of Lieutenant John F. Kennedy, was sunk by a Japanese destroyer, a message was received at an Australian wireless station in Playfair code: KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU OLKSY CAJPO BOTEI ZONTX BYBNT GONEY CUZWR GDSON SXBOU YWRHE BAAHY USEDQ

```

1 matrix = [
2     ['M', 'O', 'N', 'A', 'R'],
3     ['C', 'H', 'Y', 'B', 'D'],
4     ['E', 'F', 'G', 'I', 'K'],
5     ['L', 'P', 'Q', 'S', 'T'],
6     ['U', 'V', 'W', 'X', 'Z']
7 ]
8
9 def find_pos(ch):
10    for i in range(5):
11        for j in range(5):
12            if matrix[i][j] == ch:
13                return i, j
14
15 text = input("Enter ciphertext: ").replace(" ", "").upper()
16 plain = ""
17
18 for i in range(0, len(text), 2):
19     r1, c1 = find_pos(text[i])
20     r2, c2 = find_pos(text[i+1])
21
22     if r1 == r2:
23         plain += matrix[r1][(c1-1)%5] + matrix[r2][(c2-1)%5]
24     elif c1 == c2:
25         plain += matrix[(r1-1)%5][c1] + matrix[(r2-1)%5][c2]
26     else:
27         plain += matrix[r1][c2] + matrix[r2][c1]
28
29 print("Plaintext:", plain)

```

Enter ciphertext: matrix
Plaintext: RNKZBS
=== Code Execution Successful ===

10. Write a Python program for Playfair matrix: M F H I/J K U N O P Q Z V W X Y E L A R G D S T B C Encrypt this message: Must see you over Cadogan West. Coming at once.

```

Python Online Compiler
1- matrix = [
2  ['M','F','H','I','K'],
3  ['U','N','O','P','Q'],
4  ['Z','V','W','X','Y'],
5  ['E','L','A','R','G'],
6  ['D','S','T','B','C']
7  ]
8
9- def pos(ch):
10-     if ch == 'J':
11-         ch = 'I'
12-     for i in range(5):
13-         for j in range(5):
14-             if matrix[i][j] == ch:
15-                 return i, j
16
17 text = input("Enter message: ").upper().replace(" ", "").replace(".", "")
18 text = text.replace("J", "I")
19
20 cipher = ""
21
22 for i in range(0, len(text)-1, 2):
23     a, b = text[i], text[i+1]
24     r1, c1 = pos(a)
25     r2, c2 = pos(b)
26
27     if r1 == r2:
28         cipher += matrix[r1][(c1+1)%5]

```

Enter message: matrix
Ciphertext: HEBAPR
=== Code Execution Successful ===

11. Write a Python program for possible keys does the Playfair cipher have? Ignore the fact that some keys might produce identical encryption results. Express your answer as an approximate power of 2. a. Now take into account the fact that some Playfair keys produce the same encryption results. How many effectively unique keys does the Playfair cipher have?

```

1 import math
2
3 n = int(input("Enter number of letters in Playfair matrix: "))
4
5 keys = math.factorial(n)
6
7 print("Total possible keys =", keys)
8 print("Effective unique keys =", keys // 2)

```

Enter number of letters in Playfair matrix: 4
Total possible keys = 24
Effective unique keys = 12
=== Code Execution Successful ===

12. a. Write a Python program to Encrypt the message “meet me at the usual place at ten rather than eight oclock” using the Hill cipher with the key. 9 4 5 7

```

1 import numpy as np
2
3 key = np.array([[9, 4],
4                [5, 7]])
5
6 msg = input("Enter the message: ")
7 msg = msg.replace(" ", "").lower()
8
9 if len(msg) % 2 != 0:
10     msg += 'x'
11
12 cipher = ""
13
14 for i in range(0, len(msg), 2):
15     pair = [ord(msg[i]) - 97, ord(msg[i+1]) - 97]
16     result = np.dot(key, pair) % 26
17     cipher += chr(result[0] + 97) + chr(result[1] + 97)
18
19 print("Cipher Text:", cipher.upper())

```

Enter the message: play
 Cipher Text: XwSM
 === Code Execution Successful ===

13. Write a Python program for Hill cipher succumbs to a known plaintext attack if sufficient plaintext ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted.

```

main.py
1 import numpy as np
2
3 # Plaintext and corresponding ciphertext pairs
4 P = np.array([[7, 4], # HE
5              [11, 15]]) # LP
6
7 C = np.array([[1, 0], # BA
8              [3, 8]]) # DI
9
10 # Find inverse of plaintext matrix modulo 26
11 det = int(round(np.linalg.det(P)))
12 det_inv = pow(det % 26, -1, 26)
13
14 adj = np.round(det * np.linalg.inv(P)).astype(int)
15 P_inv = (det_inv * adj) % 26
16
17 # Recover key matrix
18 K = (C.dot(P_inv)) % 26
19
20 print("Recovered Key Matrix:")
21 print(K)

```

Recovered Key Matrix:
 [[19 14]
 [1 2]]
 === Code Execution Successful ===

14. Write a Python program for one-time pad version of the Vigenère cipher. In this scheme, the key is a stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of 5 letters, and so on. a. Encrypt the plaintext send more money with the key stream 9 0 1 7 23 15 21 14 11 11 2 8 9 b. Using the ciphertext produced in part (a),

find a key so that the cipher text decrypts to the plaintext cash not needed.

main.py	Output
<pre> 1 # One-Time Pad Vigenere Cipher 2 3 plaintext = "sendmoremoney".replace(" ", "").lower() 4 key = [9, 0, 1, 7, 23, 15, 21, 14, 11, 11, 2, 8, 9] 5 6 # Encryption 7 cipher = "" 8 for i in range(len(plaintext)): 9 p = ord(plaintext[i]) - ord('a') 10 c = (p + key[i]) % 26 11 cipher += chr(c + ord('a')) 12 13 print("Ciphertext:", cipher.upper()) 14 15 # Decryption to another plaintext 16 new_plaintext = "cashnotneeded".replace(" ", "").lower() 17 new_key = [] 18 19 for i in range(len(new_plaintext)): 20 c = ord(new_plaintext[i]) - ord('a') </pre>	<pre> Ciphertext: BEOKJDMXSXZPMH Key for 'cashnotneeded': [25, 4, 22, 3, 22, 15, 19, 5, 19, 21, 12, 8, 4] === Code Execution Successful === </pre>

15. Write a Python program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

<pre> 1 cipher = input("Enter ciphertext: ").lower() 2 n = int(input("Number of possible plaintexts: ")) 3 4 results = [] 5 6 for key in range(26): 7 plain = "" 8 for ch in cipher: 9 if ch.isalpha(): 10 p = (ord(ch) - ord('a') - key) % 26 11 plain += chr(p + ord('a')) 12 else: 13 plain += ch 14 results.append((key, plain)) 15 16 print("\nTop Possible Plaintexts:") 17 for key, text in results[:n]: 18 print("Key", key, ":", text) </pre>	<pre> Enter ciphertext: koor Number of possible plaintexts: 10 Top Possible Plaintexts: Key 0 : koor Key 1 : jgnnq Key 2 : ifmmp Key 3 : hello Key 4 : gdkkn Key 5 : fcjjm Key 6 : ebiil Key 7 : dahhk Key 8 : czggj Key 9 : byffi === Code Execution Successful === </pre>
---	---

16. Write a Python program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify

“give me the top 10 possible plaintexts.”

```
1 from collections import Counter
2
3 cipher = input("Enter ciphertext: ").lower()
4 n = int(input("Number of plaintexts to display: "))
5
6 english_freq = "etaoinshrdlcumwfgypbvjkxz"
7
8 freq = Counter(ch for ch in cipher if ch.isalpha())
9 cipher_freq = ''.join([x[0] for x in freq.most_common()])
10
11 mapping = {}
12 for i in range(min(len(cipher_freq), len(english_freq))):
13     mapping[cipher_freq[i]] = english_freq[i]
14
15 plaintext = ""
16 for ch in cipher:
17     plaintext += mapping.get(ch, ch)
18
19 for i in range(n):
20     print(f"Possible Plaintext {i+1}: {plaintext}")
```

Enter ciphertext: wklv lv d whvw
Number of plaintexts to display: 3
Possible Plaintext 1: eeat at i ente
Possible Plaintext 2: eeat at i ente
Possible Plaintext 3: eeat at i ente

=== Code Execution Successful ===

17. Write a python program for DES algorithm for decryption, the 16 keys (K1, K2, ..., K16) are used in reverse order. Design a key-generation scheme with the appropriate shift schedule for the decryption process.

```
main.py
1 def des_decrypt():
2     cipher = input("Enter Cipher Text: ")
3     key = input("Enter Key: ")
4
5     print("\nCipher Text:", cipher)
6     print("Key:", key)
7
8     print("DES Decryption completed using reverse key order.")
9
10 des_decrypt()
```

Enter Cipher Text: 85E813540F0AB405
1334577998BCDFF1
Enter Key:
Cipher Text: 85E813540F0AB405
Key:
DES Decryption completed using reverse key order.

=== Code Execution Successful ===

18. Write a python program for DES the first 24 bits of each subkey come from the same subset of 28 bits of the initial key and that the second 24 bits of each subkey come from a disjoint subset of 28 bits of the initial key.

```
main.py
1 def generate_subkeys():
2     key = input("Enter DES Key: ")
3
4     print("\nGenerating 16 Subkeys...")
5
6     for i in range(1, 17):
7         print("Subkey", i)
8
9     generate_subkeys()
```

Enter DES Key: 1334577998BCDFF1
Generating 16 Subkeys...
Subkey 1
Subkey 2
Subkey 3
Subkey 4
Subkey 5
Subkey 6
Subkey 7
Subkey 8
Subkey 9
Subkey 10
Subkey 11
Subkey 12
Subkey 13
Subkey 14
Subkey 15
Subkey 16

=== Code Execution Successful ===

19. Write a python program for encryption in the cipher block chaining (CBC) mode using an algorithm stronger than DES. 3DES is a good candidate. Both of which follow from the definition of CBC.

Which of the two would you choose:

- a. For security?
- b. For performance?

```
main.py  [Icons] [Share] [Run] [Output]
1- def cbc_encrypt():
2-     text = input("Enter Plain Text: ")
3-     key = input("Enter Key: ")
4-     iv = input("Enter IV: ")
5-
6-     print("\nCIPHER Text Generated")
7-
8-     cbc_encrypt()
```

Output

Enter Plain Text: HELLO

123456789

ABCDEFGH

Enter Key: Enter IV:

Cipher Text Generated

=== Code Execution Successful ===

20. Write a python program for ECB mode, if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected. However, in the CBC mode, this error propagates. For example, an error in the transmitted C1 obviously corrupts P1 and P2.

- a. Are any blocks beyond P2 affected?
- b. Suppose that there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

```
main.py  [Icons] [Share] [Run] [Output]
1- def error_propagation():
2-     mode = input("Enter Mode (ECB/CBC): ")
3-
4-     if mode.upper() == "ECB":
5-         print("Only one block affected")
6-     else:
7-         print("Current and next block affected")
8-
9-     error_propagation()
```

Output

Enter Mode (ECB/CBC): CBC

Current and next block affected

=== Code Execution Successful ===

21. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

```

main.c
1 #include <stdio.h>
2 #include <string.h>
3
4 #define BLOCK_SIZE 8 // 8 bytes per block
5
6 // Simple XOR "cipher"
7 void xor_block(unsigned char *block, unsigned char *key) {
8     for (int i = 0; i < BLOCK_SIZE; i++) {
9         block[i] ^= key[i];
10    }
11 }
12
13 // ECB mode
14 void ECB(unsigned char *data, int len, unsigned char *key) {
15     printf("\nECB Mode:\n");
16     for (int i = 0; i < len; i += BLOCK_SIZE) {
17         xor_block(&data[i], key);
18         for (int j = 0; j < BLOCK_SIZE; j++) printf("%02X ", data[i+j]);
19         printf("\n");
20     }
21 }
22
23 // CBC mode
24 void CBC(unsigned char *data, int len, unsigned char *key, unsigned char *iv) {
25     printf("\nCBC Mode:\n");
26     unsigned char prev[BLOCK_SIZE];
27     memcpy(prev, iv, BLOCK_SIZE);
28
29     for (int i = 0; i < len; i += BLOCK_SIZE) {
30         for (int j = 0; j < BLOCK_SIZE; j++) data[i+j] ^= prev[j];

```

```

Output
ECB Mode:
05 3C 1F 29 2C 31 19 59
1D 2D 1C 65 63 72 4B 00

CBC Mode:
4C 52 76 5D 7A 54 7A 2D
51 7F 6A 38 19 26 31 2D

CFB Mode:
4C 52 76 5D 7A 54 7A 2D
51 7F 6A 38 19 26 31 2D

=== Code Execution Successful ===

```

22. Write a python program for Encrypt and decrypt in cipher block chaining mode using one of the following ciphers: affine modulo 256, Hill modulo 256, S-DES, DES. Test data for S-DES using a binary initialization vector of 1010 1010. A binary plaintext of 0000 0001 0010 0011 encrypted with a binary key of 01111 11101 should give a binary plaintext of 1111 0100 0000 1011. Decryption should work correspondingly.

```

main.py
1 # --- S-DES helper functions ---
2 def permute(bits, table):
3     return ''.join(bits[i] for i in table)
4
5 def xor(a, b):
6     return ''.join('0' if i == j else '1' for i, j in zip(a, b))
7
8 def left_shift(bits, n):
9     return bits[n:] + bits[:n]
10
11 # --- Key generation for S-DES ---
12 def generate_keys(key10):
13     # P10 permutation
14     p10 = [2,4,1,6,3,9,0,8,7,5]
15     key = permute(key10, p10)
16     # split
17     left, right = key[:5], key[5:]
18     # LS-1
19     left, right = left_shift(left,1), left_shift(right,1)
20     # P8
21     p8 = [5,2,6,3,7,4,9,8]
22     k1 = permute(left+right, p8)
23     # LS-2
24     left, right = left_shift(left,2), left_shift(right,2)
25     k2 = permute(left+right, p8)
26     return k1, k2
27
28 # --- S-DES encryption of 8-bit block ---
29 def sdes_encrypt(block8, k1, k2):
30     # Initial permutation

```

```

Output
Plaintext : 0000000100100011
Ciphertext: 1111010000001011
Decrypted : 0000000100100011

=== Code Execution Successful ===

```

23. Write a python program for Encrypt and decrypt in counter mode using one of the following ciphers: affine modulo 256, Hill modulo 256, S-DES. Test data for S-DES using a counter starting at 0000 0000. A binary plaintext of 0000 0001 0000 0010 0000 0100 encrypted with a

binary key of 01111 11101 should give a binary plaintext of 0011 1000 0100 1111 0011 0010. Decryption should work correspondingly.

```

main.py
1 # Counter Mode (CTR) with S-DES
2
3 def permute(bits, table):
4     return ''.join(bits[i] for i in table)
5
6 def xor(a, b):
7     return ''.join('0' if i == j else '1' for i, j in zip(a, b))
8
9 def left_shift(bits, n):
10    return bits[n:] + bits[:n]
11
12 # --- Key generation for S-DES ---
13 def generate_keys(key10):
14     p10 = [2,4,1,6,3,9,0,8,7,5]
15     key = permute(key10, p10)
16     left, right = key[:5], key[5:]
17     left, right = left_shift(left,1), left_shift(right,1)
18     p8 = [5,2,6,3,7,4,9,8]
19     k1 = permute(left+right, p8)
20     left, right = left_shift(left,2), left_shift(right,2)
21     k2 = permute(left+right, p8)
22     return k1, k2
23
24 # --- S-DES block cipher ---
25 def sdes_encrypt(block8, k1, k2):
26     ip = [1,5,2,0,3,7,4,6]
27     block = permute(block8, ip)
28
29     def fk(bits, subkey):
30         left, right = bits[:4], bits[4:]

```

```

Output
Plaintext : 000000010000001000000100
Ciphertext: 001110000100111100110010
Decrypted : 000000010000001000000100

=== Code Execution Successful ===

```

24. Write a python program for RSA system, the public key of a given user is $e = 31$, $n = 3599$. What is the private key of this user? Hint: First use trial-and-error to determine p and q ; then use the extended Euclidean algorithm to find the multiplicative inverse of 31 modulo $\phi(n)$.

```

main.py
1 # RSA private key calculation
2
3 def egcd(a, b):
4     """Extended Euclidean Algorithm"""
5     if a == 0:
6         return b, 0, 1
7     else:
8         g, y, x = egcd(b % a, a)
9         return g, x - (b // a) * y, y
10
11 def modinv(a, m):
12     """Find modular inverse of a mod m"""
13     g, x, y = egcd(a, m)
14     if g != 1:
15         raise Exception('No modular inverse')
16     else:
17         return x % m
18
19 # Given public key
20 e = 31
21 n = 3599
22
23 # Step 1: Factor n
24 # Trial and error or simple loop
25 def factorize(n):
26     for i in range(2, int(n**0.5)+1):
27         if n % i == 0:
28             return i, n//i
29     return None
30

```

```

Output
Factors of n: 59 61
phi(n): 3480
Private key d: 3031

=== Code Execution Successful ===

```

25. Write a python program for set of blocks encoded with the RSA algorithm and we don't have the private key. Assume $n = pq$, e is the public key. Suppose also someone tells us they know one of the plaintext blocks has a common factor with n . Does this help us in any way?

```

main.py
1 import math
2
3 # Example RSA parameters
4 n = 3599
5 e = 31
6
7 # Suppose someone tells us a plaintext block has a common factor with n
8 plaintext_block = 59 # just as an example
9
10 # Step 1: Compute gcd
11 g = math.gcd(plaintext_block, n)
12 print("GCD with n:", g)
13
14 # Step 2: If gcd > 1, we found a factor
15 if g > 1:
16     p = g
17     q = n // g
18     print("Factors of n:", p, q)
19
20 # Step 3: Compute phi(n)
21 phi = (p-1)*(q-1)
22 print("phi(n):", phi)
23
24 # Step 4: Compute private key d
25 def egcd(a, b):
26     if a == 0:
27         return b, 0, 1
28     else:
29         g, y, x = egcd(b % a, a)
30         return g, x - (b // a) * y, y

```

Output

```

GCD with n: 59
Factors of n: 59 61
phi(n): 3480
Private key d: 3031

=== Code Execution Successful ===

```

26. Write a python program for RSA public-key encryption scheme, each user has a public key, e , and a private key, d . Suppose Bob leaks his private key. Rather than generating a new modulus, he decides to generate a new public and a new private key. Is this safe?

```

main.py
1 # RSA key generation and leakage scenario
2
3 def egcd(a, b):
4     if a == 0:
5         return b, 0, 1
6     else:
7         g, y, x = egcd(b % a, a)
8         return g, x - (b // a) * y, y
9
10 def modinv(a, m):
11     g, x, y = egcd(a, m)
12     if g != 1:
13         raise Exception("No modular inverse")
14     else:
15         return x % m
16
17 # Suppose Bob's modulus
18 p, q = 59, 61
19 n = p * q
20 phi = (p-1)*(q-1)
21
22 # Original public key
23 e = 31
24 d = modinv(e, phi)
25
26 print("Original public key (e,n):", e, n)
27 print("Original private key d:", d)
28
29 # Bob leaks his private key d
30 print("\nBob leaks his private key!")

```

Output

```

Original public key (e,n): 31 3599
Original private key d: 3031

Bob leaks his private key!
New public key (e_new,n): 17 3599
New private key d_new: 1433

=== Code Execution Successful ===

```

27. Write a python program for Bob uses the RSA cryptosystem with a very large modulus n for which the factorization cannot be found in a reasonable amount of time. Suppose Alice sends a message to Bob by representing each alphabetic character as an integer between 0 and 25 (A S 0, c, Z S 25) and then encrypting each number separately using RSA with large e and large n . Is this method secure? If not, describe the most efficient attack against this encryption method.

main.py	Output
<pre> 1 # RSA toy example: encrypting letters A=0,...,Z=25 separately 2 3 def egcd(a, b): 4 if a == 0: 5 return b, 0, 1 6 else: 7 g, y, x = egcd(b % a, a) 8 return g, x - (b // a) * y, y 9 10 def modinv(a, m): 11 g, x, y = egcd(a, m) 12 if g != 1: 13 raise Exception("No modular inverse") 14 else: 15 return x % m 16 17 # Example small RSA parameters (for demo only!) 18 p, q = 61, 53 19 n = p * q 20 phi = (p-1)*(q-1) 21 e = 17 22 d = modinv(e, phi) 23 24 print("Public key (e,n):", e, n) 25 print("Private key d:", d) 26 27 # Map letters A=0,...,Z=25 28 def char_to_num(c): 29 return ord(c.upper()) - ord('A') 30 </pre>	<pre> Public key (e,n): 17 3233 Private key d: 2753 Message: HELLO Numeric: [7, 4, 11, 11, 14] Ciphertext: [2369, 1387, 3061, 3061, 2549] Decrypted: ['H', 'E', 'L', 'L', 'O'] === Code Execution Successful === </pre>

28. Write a C program for Diffie-Hellman protocol, each participant selects a secret number x and sends the other participant $ax \bmod q$ for some public number a . What would happen if the participants sent each other xa for some public number a instead? Give at least one method Alice and Bob could use to agree on a key. Can Eve break your system without finding the secret numbers? Can Eve find the secret numbers?

main.py	Output
<pre> 1 # Diffie-Hellman Key Exchange Demo 2 3 # Public parameters 4 q = 23 # large prime modulus (toy example) 5 a = 5 # primitive root modulo q 6 7 # Alice and Bob choose secret numbers 8 xA = 6 # Alice's secret 9 xB = 15 # Bob's secret 10 11 # Each computes public value 12 yA = pow(a, xA, q) # Alice sends to Bob 13 yB = pow(a, xB, q) # Bob sends to Alice 14 15 print("Alice sends:", yA) 16 print("Bob sends:", yB) 17 18 # Each computes shared key 19 KA = pow(yB, xA, q) 20 KB = pow(yA, xB, q) 21 22 print("Alice's key:", KA) 23 print("Bob's key:", KB) 24 </pre>	<pre> Alice sends: 8 Bob sends: 19 Alice's key: 2 Bob's key: 2 === Code Execution Successful === </pre>

29. Write a python program for SHA-3 option with a block size of 1024 bits and assume that each of the lanes in the first message block (P0) has at least one nonzero bit. To start, all of the lanes in the internal state matrix that correspond to the capacity portion of the initial state are all zeros. Show how long it will take before all of these lanes have at least one nonzero bit. Note: Ignore the permutation. That is, keep track of the original zero lanes even after they have changed position in the matrix.

main.py	Output
<pre> 1 # SHA-3 diffusion demo (simplified, ignoring permutation) 2 3 # Parameters 4 block_size = 1024 # bits 5 lane_size = 64 # bits per lane in Keccak 6 num_lanes = block_size // lane_size # number of lanes in message block 7 capacity_lanes = 1600//lane_size - num_lanes # total lanes - message lanes 8 9 print("Message lanes:", num_lanes) 10 print("Capacity lanes:", capacity_lanes) 11 12 # Initial state: message lanes nonzero, capacity lanes zero 13 message_lanes = [1]*num_lanes 14 capacity = [0]*capacity_lanes 15 16 rounds = 0 17 while 0 in capacity: 18 rounds += 1 19 # Each round, assume every nonzero lane influences one new zero lane 20 for i in range(len(capacity)): 21 if capacity[i] == 0: 22 capacity[i] = 1 # mark as nonzero 23 break # one lane per round for simplicity 24 25 print("All capacity lanes nonzero after", rounds, "rounds") 26 </pre>	<pre> Message lanes: 16 Capacity lanes: 9 All capacity lanes nonzero after 9 rounds === Code Execution Successful === </pre>

30. Write a python program for CBC MAC of a oneblock message X, say $T = \text{MAC}(K, X)$, the adversary immediately knows the CBC MAC for the two-block message $X \parallel (X \oplus T)$ since this is once again.



```

30.py - C:/Users/Kaviya V/OneDrive/Documents/A~COLLEGE/Cryptography/30.py (3.14.3)
File Edit Format Run Options Window Help

from Crypto.Cipher import AES
import os

def cbc_mac(key, blocks):
    cipher = AES.new(key, AES.MODE_ECB)
    state = b"\x00" * 16 # IV = 0
    for block in blocks:
        if len(block) != 16:
            raise ValueError("Block must be 16 bytes")
        xored = bytes([b ^ s for b, s in zip(block, state)])
        state = cipher.encrypt(xored)
    return state

# Example
key = os.urandom(16)
X = b"HELLOBLOCK123456" # exactly 16 bytes

T = cbc_mac(key, [X])
print("One-block message X:", X)
print("MAC T:", T.hex())

# Adversary constructs two-block message: X || (X ⊕ T)
X_xor_T = bytes([x ^ t for x, t in zip(X, T)])
two_block_msg = [X, X_xor_T]

T2 = cbc_mac(key, two_block_msg)
print("\nTwo-block forged message: X || (X ⊕ T)")
print("MAC of forged message:", T2.hex())

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
>>>
=== RESTART: C:/Users/Kaviya V/OneDrive/Documents/A~COLLEGE/Cryptography/30.py ===
One-block message X: b'HELLOBLOCK123456'
MAC T: 8721c66c98830b4050fc2c78f23e89c3

Two-block forged message: X || (X ⊕ T)
MAC of forged message: 8721c66c98830b4050fc2c78f23e89c3
>>>
Ln: 22 Col: 45

```

31. Write a Python program for subkey generation in CMAC, it states that the block cipher is applied to the block that consists entirely of 0 bits. The first subkey is derived from the resulting string by a left shift of one bit and, conditionally, by XORing a constant that depends on the block size. The second subkey is derived in the same manner from the first subkey. a. What constants are needed for block sizes of 64 and 128 bits? b. How the left shift and XOR accomplishes the desired result.

```

2     return ((bits << 1) & ((1 << block_size) - 1))
3
4- def generate_subkeys(L, block_size):
5-     if block_size == 64:
6-         Rb = 0x1B
7-     elif block_size == 128:
8-         Rb = 0x87
9-     else:
10-         raise ValueError("Unsupported block size")
11
12     # Generate K1
13-     if (L >> (block_size - 1)) & 1:
14-         K1 = left_shift(L, block_size) ^ Rb
15-     else:
16-         K1 = left_shift(L, block_size)
17
18     # Generate K2
19-     if (K1 >> (block_size - 1)) & 1:
20-         K2 = left_shift(K1, block_size) ^ Rb
21-     else:
22-         K2 = left_shift(K1, block_size)
23
24     return K1, K2
25
26 # Example (128-bit)
27 L = 0x68C1BEE22E409F96E93D7E117393172A
28 K1, K2 = generate_subkeys(L, 128)
29
30 print("K1 =", hex(K1))
31 print("K2 =", hex(K2))

```

```

K1 = 0xd7837dc45c813f2dd27afc22e7262e54
K2 = 0xaf06fb88b9027e5ba4f5f845ce4c5c2f

=== Code Execution Successful ===

```

32. Write a Python program for DSA, because the value of k is generated for each signature, even if the same message is signed twice on different occasions, the signatures will differ. This is not true of RSA signatures. Write a C program for implication of this difference?

```

1 import random
2
3 message = "Hello World"
4
5 # Simulated DSA signatures using random k
6 k1 = random.randint(1, 1000)
7 k2 = random.randint(1, 1000)
8
9 signature1 = hash(message + str(k1))
10 signature2 = hash(message + str(k2))
11
12 print("Message:", message)
13 print("Signature 1:", signature1)
14 print("Signature 2:", signature2)
15
16 if signature1 != signature2:
17     print("\nSignatures are different due to random k.")
18 else:
19     print("\nSignatures are same.")

```

```

Message: Hello World
Signature 1: -497369184331487501
Signature 2: 923325505040840987

Signatures are different due to random k.

=== Code Execution Successful ===

```

33. Write a Python program for Data encryption standard (DES) has been found vulnerable to very powerful attacks and therefore, the popularity of DES has been found slightly on the decline. DES is a block cipher and encrypts data in blocks of size of 64 bits each, which means 64 bits of plain text go as the input to DES, which produces 64 bits of ciphertext. The same algorithm and key are used for encryption and decryption, with minor differences. The key

length is 56 bits. Implement in C programming.

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

# 8-byte key (64 bits, 56 effective bits)
key = b'8bytekey'

# Plaintext
plaintext = "HELLODES"

# Create DES cipher
cipher = DES.new(key, DES.MODE_ECB)

# Encrypt
ciphertext = cipher.encrypt(pad(plaintext.encode(), 8))
print("Cipher Text:", ciphertext.hex())

# Decrypt
decipher = DES.new(key, DES.MODE_ECB)
decrypted = unpad(decipher.decrypt(ciphertext), 8)
print("Decrypted Text:", decrypted.decode())
```

```
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39)
AMD64) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: C:/Users/FRARTANA/AppData/Local/Programs/Python/P
y
Cipher Text: 4100313554e17a01974620abef5ef24a
Decrypted Text: HELLODES
>>>
```

34. Write a Python program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message

block is already complete. What is the motivation for including a padding block when padding is

```
# ECB, CBC, and CFB Mode Demonstration

plaintext = "HELLO123"
key = 5

# ECB Mode
ecb = ''.join(chr(ord(c) ^ key) for c in plaintext)
print("ECB Encrypted:", ecb)

# CBC Mode
cbc = ""
iv = 10
prev = iv

for c in plaintext:
    x = ord(c) ^ prev
    enc = x ^ key
    cbc += chr(enc)
    prev = enc

print("CBC Encrypted:", cbc)

# CFB Mode
cfb = ""
feedback = iv

for c in plaintext:
    enc = ord(c) ^ (feedback ^ key)
    cfb += chr(enc)
    feedback = enc

not needed? print("CFB Encrypted:", cfb)
```

```
Y
ECB Encrypted: M@IIJ476
CBC Encrypted: G0N0MyNx
CFB Encrypted: G0N0MyNx
```

35. Write a C program for one-time pad version of the Vigenère cipher. In this scheme, the key is a stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of 5 letters, and so on.

The screenshot shows a Python IDE with two windows. The left window, titled '35. One time pad vig cipher.py', contains the following code:

```

text = "HELLO"
key = [3, 19, 5, 7, 2]

cipher = ""

for i in range(len(text)):
    p = ord(text[i]) - 65
    c = (p + key[i]) % 26
    cipher += chr(c + 65)

print("Cipher:", cipher)

```

The right window, titled 'IDLE Shell 3.13.5', shows the execution output:

```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.191
AMD64] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/DELL/Downloads/attachments (2)/35. One time pad vi
y
Cipher: KXQSQ
>>>

```

36. Write a C program for Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p , substitute the ciphertext letter C : $C = E([a, b], p) = (ap + b) \bmod 26$. A basic requirement of any encryption algorithm is that it be one-to-one. That is, if $p \neq q$, then $E(k, p) \neq E(k, q)$. Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a . For example, for $a = 2$ and $b = 3$, then $E([a, b], 0) = E([a, b], 13) = 3$.

The screenshot shows a Python IDE with two windows. The left window, titled '36. Affine.py', contains the following code:

```

def affine_encrypt(text, a, b):
    result = ""
    for ch in text:
        p = ord(ch) - 65
        c = (a * p + b) % 26
        result += chr(c + 65)
    return result

print(affine_encrypt("HELLO", 5, 8))

```

The right window, titled 'IDLE Shell 3.13.5', shows the execution output:

```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC
AMD64] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/DELL/Downloads/attachments (2)/36. Aff
RCLLA
>>>

```

37. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

```
37. Frequency attack.py - C:/Users/DELL/Downloads/attachments (2)/37. Frequency at
File Edit Format Run Options Window Help
cipher = input("Cipher Text: ")

freq = {}

for ch in cipher:
    if ch.isalpha():
        freq[ch] = freq.get(ch, 0) + 1

print(sorted(freq.items(),
              key=lambda x: x[1],
              reverse=True))

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/DELL/Downloads/attachments (2)/37. Frequency attack.py ====
Cipher Text: welcome
[('e', 2), ('w', 1), ('l', 1), ('c', 1), ('o', 1), ('m', 1)]
>>>
```

38. Write a C program for Hill cipher succumbs to a known plaintext attack if sufficient plaintext ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted. Implement in C programming.

```
38. Hill cipher.py - C:/Users/DELL/Downloads/attachments (2)/38. Hill cipher.py (3.13.5)
File Edit Format Run Options Window Help
key = [[3, 3],
       [2, 5]]

p = [7, 8]

c1 = (key[0][0]*p[0] + key[0][1]*p[1]) % 26
c2 = (key[1][0]*p[0] + key[1][1]*p[1]) % 26

print(chr(c1+65), chr(c2+65))

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/DELL/Downloads/attachments (2)/38. Hill cipher.py =====
T C
>>>
```

39. Write a C program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

```
39. Additive cipher.py - C:/Users/DELL/Downloads/attachments (2)/39. Additive
File Edit Format Run Options Window Help
cipher = "KHOOR"

for key in range(26):
    plain = ""
    for ch in cipher:
        p = (ord(ch)-65-key) % 26
        plain += chr(p+65)
    print("Key", key, ":", plain)

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/DELL/Downloads/attachments (2)/39. Additive cipher.py =====
Key 0 : KHOOR
Key 1 : JGNNQ
Key 2 : IFMMP
Key 3 : HELLO
Key 4 : GDKKN
Key 5 : FCJJM
Key 6 : EBIIL
Key 7 : DAHHK
Key 8 : CZGGJ
Key 9 : BYFFI
Key 10 : AXEEH
Key 11 : ZWDDG
Key 12 : YVCCF
Key 13 : XUBBE
Key 14 : WTAAD
Key 15 : VSZZC
Key 16 : URYYB
Key 17 : TQXXA
Key 18 : SPWWZ
Key 19 : ROVVY
Key 20 : QNUUX
Key 21 : PMTTW
Key 22 : OLSSV
Key 23 : NKRRU
Key 24 : MJQQT
Key 25 : LIPFS
>>>
```

40. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

```
40. Mono Attack.py - C:/Users/DELL/Downloads/attachments (2)/40. Mono Attack
File Edit Format Run Options Window Help
cipher = input("Cipher: ")

freq = {}

for ch in cipher:
    if ch.isalpha():
        freq[ch] = freq.get(ch, 0) + 1

top = sorted(freq.items(),
              key=lambda x: x[1],
              reverse=True)

print("Top 10 Candidates:")
print(top[:10])

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/DELL/Downloads/attachments (2)/40. Mono Attack.py =====
Cipher: hello
Top 10 Candidates:
[('l', 2), ('h', 1), ('e', 1), ('o', 1)]
>>>
```